Week_6 - Assignments

Data structures and Fundamental Algorithm :-

(1). For the graph in Figure 9.81, choose one appropriate representation and show the graph in that representation.

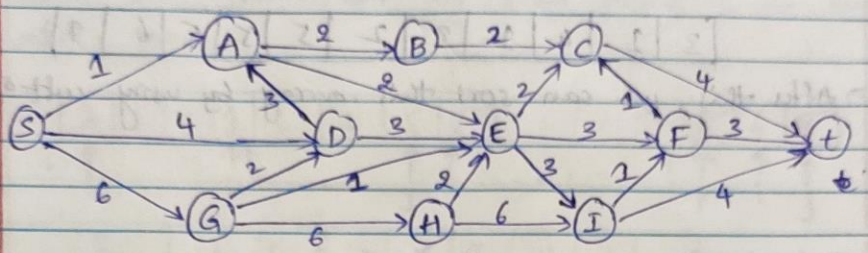


Fig : 9.81 : Directed Acyclic Graph (DAG)

Ans:- **Representation of Graph :-**

→ ~~Space required~~ $O(|V|^2)$.

→ Given graph is directed acyclic graph because it has the directions but not even a single cycle is forming so it is called as DAG.

→ It is a weighted graph, because it has the values (weights) on edges.

→ We have the different types of ways to represent a graph.

→ Here I am using "Adjacent list" method to represent the given graph.

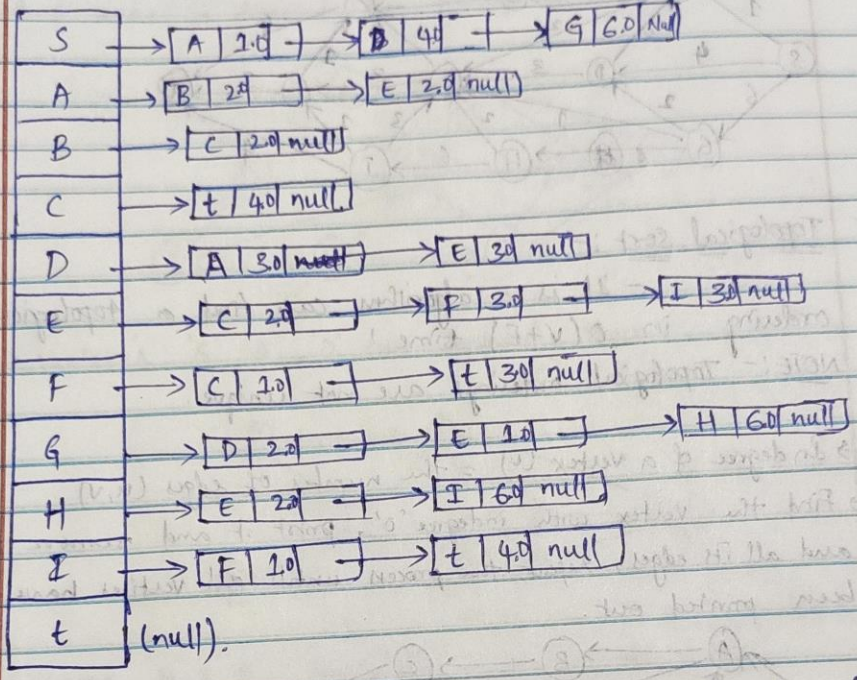
*) **Adjacent List :-**

→ For each vertex; create a list for all adjacent vertices.

→ Space required $\Theta(|E|+|V|)$

→ Good for sparsh graph.

Adj Info



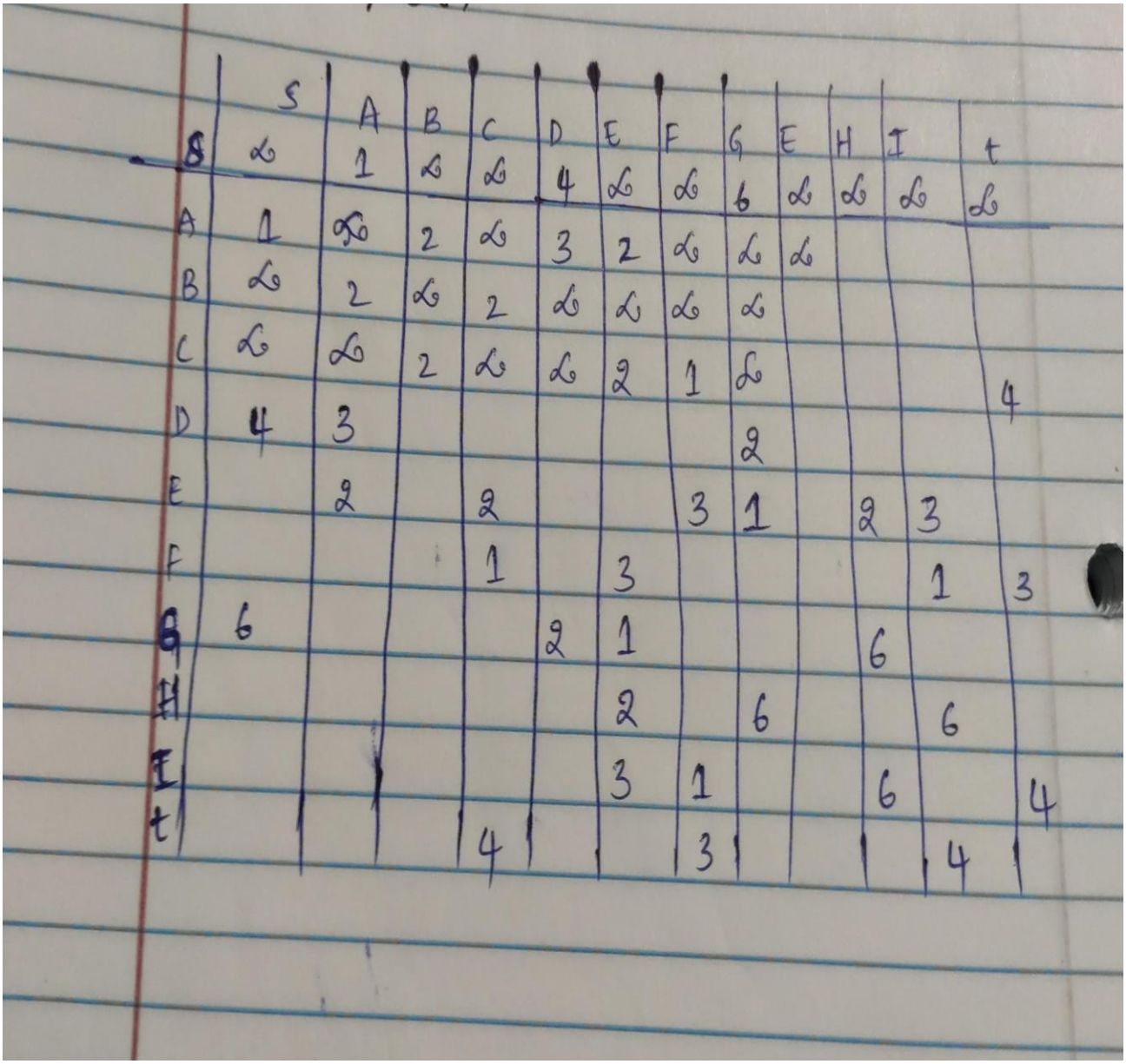
→ Here, adj info - array is $|V|$ and the adjacent nodes $|E|$. So the sum of them gives, space required.

→ $|E| << |V|^2$

→ Here, I have used adjacent list method, because other than 't' vertex, every vertex has outgoing edge and it is simple. but, where as in the case of adjacent matrix method, the matrix is going to be very huge and has more number of '0' than the actual weights. so i thought this matrix is going to be consise and suitable.

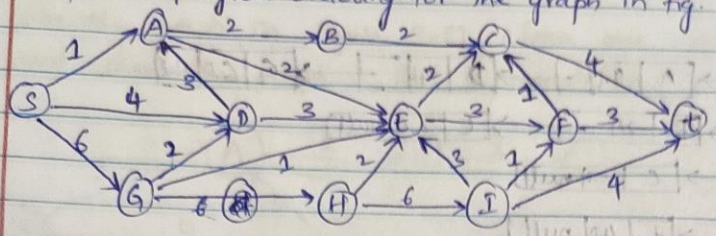
#

So, here I am attaching the table for adjacent matrix method.



	S	A	B	C	D	E	F	G	E	H	I	t
S	∞	1	∞	∞	4	∞	∞	6	∞	∞	∞	∞
A	1	∞	2	∞	3	2	∞	∞	∞			
B	∞	2	∞	2	∞	∞	∞	∞				
C	∞	∞	2	∞	∞	2	1	∞				4
D	4	3						2				
E		2		2			3	1		2	3	
F				1		3					1	3
G	6				2	1				6		
H						2		6			6	
I						3	1			6		4
t				4			3				4	

9.1) Find a topological ordering for the graph in fig. 9.81.



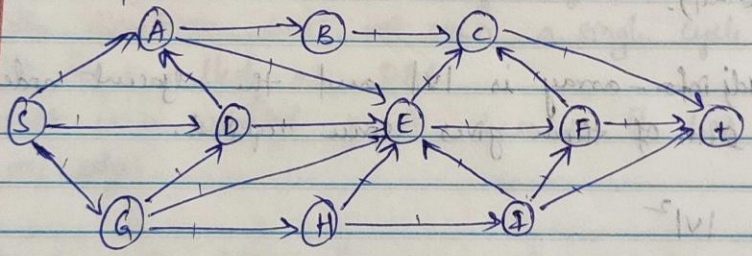
Ans:- Topological sort :-

- It is a algorithm can find a topological ordering in $O(V+E)$ time!

NOTE:- Topological orderings are not unique.

→ In degree of a vertex (v) = the number of edges (u,v).

→ Find the vertex with indegree '0', print it and remove it and all its edges, Repeat the process untill all vertices have been printed out.



→ First we need to start with any node that has no incoming edges, here the node with no incoming edges is node 's'.

→ Add the node with no incoming edges to the stack. i.e 's'

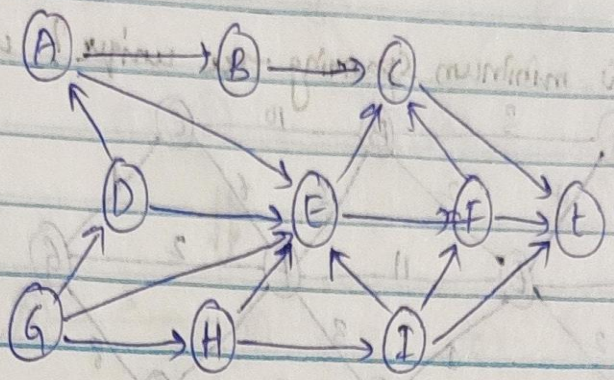
→ Remove the node from the stack i.e 's'

→ Repeat these steps untill last vertex.

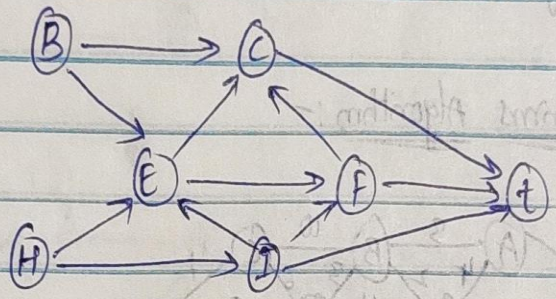
→ Here indegree = 21 = number of edges.

So, the major steps are:

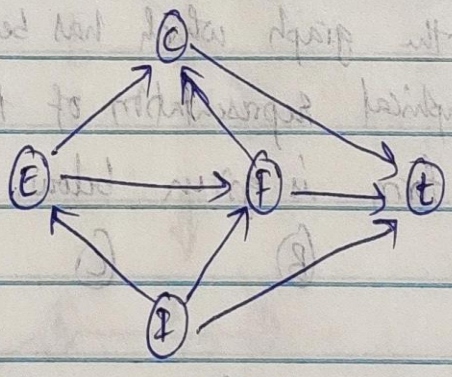
→ 's' vertex is dequeued from the graph so,



→ Vertexes 'G', 'A', 'D' are dequeued then.



→ Vertex, 'B' & 'H' removed then,



→ Vertex 'E' dequeued then

Vertex:	Indegree Before dequeue #																			
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
S	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
A	2	1	0	0	0	0	0	0	0	0	0	0	→								
B	1	1	1	0	0	0	0	0	0	0	0	0									
C	3	3	3	3	3	3	2	1	0	0	0	0	→								
D	2	1	0	0	0	0	0	0	0	0	0	0									
E	5	5	4	3	2	1	0	0	0	0	0	0	→								
F	2	2	2	2	2	2	1	0	0	0	0	0	→								
G	1	0	0	0	0	0	0	0	0	0	0	0	→								
H	1	1	0	0	0	0	0	0	0	0	0	0									
I	1	1	1	1	1	0	0	0	0	0	0	0	→								
t	3	3	3	3	3	3	3	3	3	2	1	0	→								

Enqueue: S G A,D,H,B I E C,F t

Dequeue: S G A D H B I E C F t

Topological ordering is: S G A D H B I E C F t

9.15) (a) Find a minimum spanning tree for the graph in Figure 9.84 using both prim's and Kruskal's algorithms.

(b) Is this minimum spanning tree unique? why?

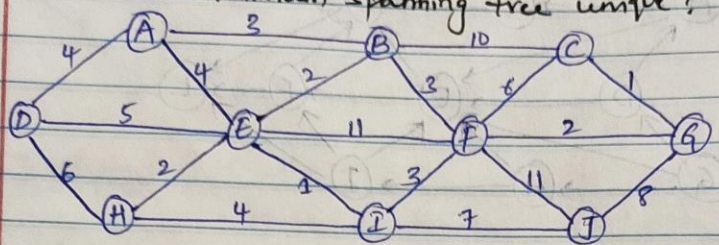
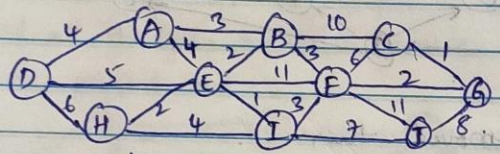
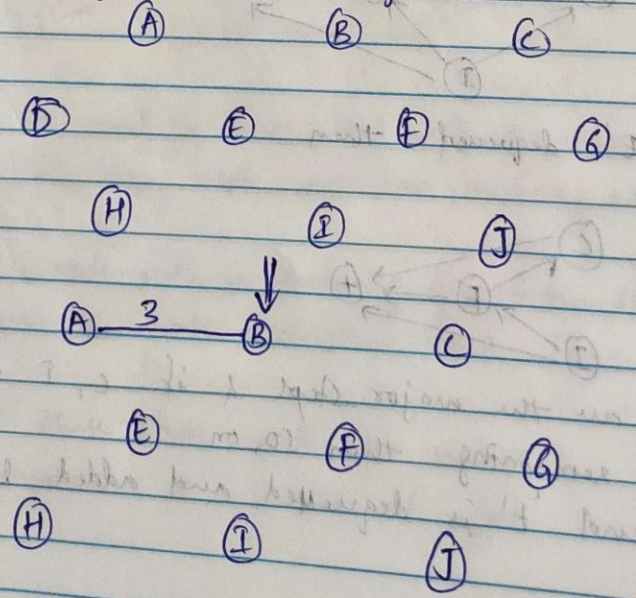


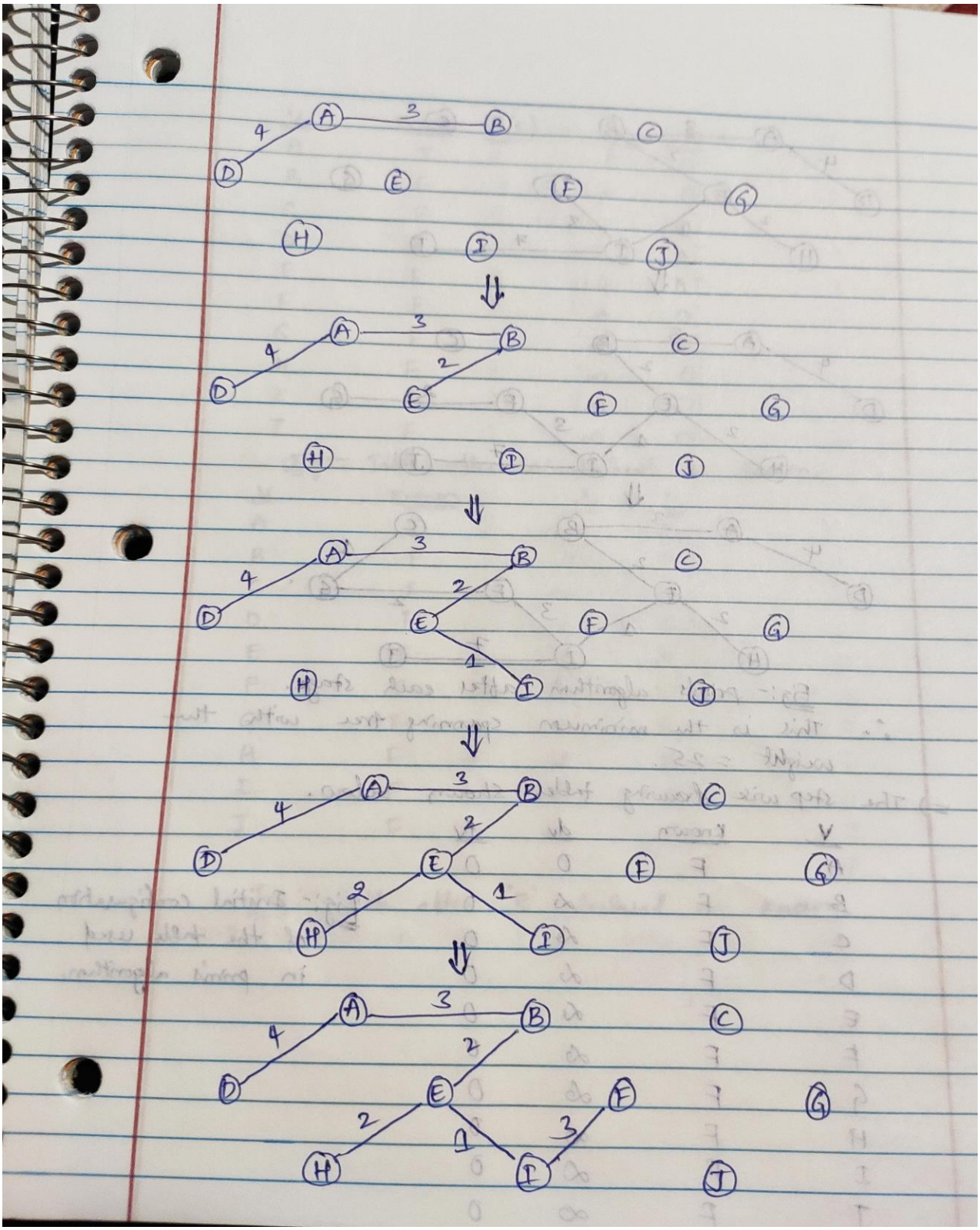
Fig: 9.84: Graph.

(a). Using prims Algorithm :-



→ This is the graph which has been given so, The step wise graphical representation of how minimum spanning tree is going to form is given below:




A
B
C
D
E
F
G
H
I
J
4
3
2
1
2
3

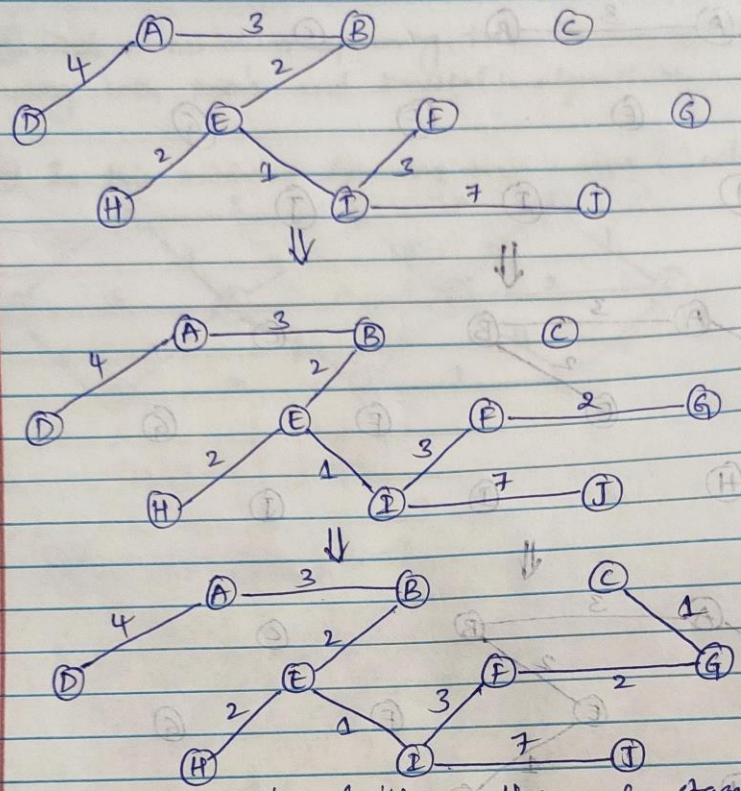


Fig:- prim's algorithm after each stage.

∴ This is the minimum spanning tree with the weight = 25.

⇒ The step wise drawing tables shown below.

V	known	dv	pv
A	F	0	0
B	F	∞	0
C	F	∞	0
D	F	∞	0
E	F	∞	0
F	F	∞	0
G	F	∞	0
H	F	∞	0
I	F	∞	0
J	F	∞	0

Fig:- Initial configuration of the table used in prim's algorithm.

V	Known (T/F)	dv	Pv
A	T	0	0
B	F	3	A
C	F	∞	0
D	F	4	A
E	F	4	A
F	F	∞	0
G	F	∞	0
H	F	∞	0
I	F	∞	0
J	F	∞	0

Fig:- Table after 'A' declared as known.

V	known	dv	Pv
A	T	0	0
B	F	2	E
C	F	∞	0
D	F	4	A
E	T	4	A
F	F	11	E
G	F	∞	0
H	F	2	E
I	F	1	E
J	F	∞	0

Fig:- Table after 'E' declared as known.

V	known	dv	Pv
A	T	0	0
B	T	2	E
C	T	10	B
D	T	4	A
E	T	4	A
F	F	3	B
G	F	1	C
H	F	2	E
I	F	1	E
J	F	∞	0

Fig:- Table after B, C, D declared as known.

V	known	dv	Pv
A	T	0	0
B	T	2	E
C	T	10	B
D	T	4	A
E	T	4	A
F	F	3	I
G	F	2	E
H	T	2	E
I	T	1	E
J	F	7	I

Fig:- Table after H, I are declared as known.

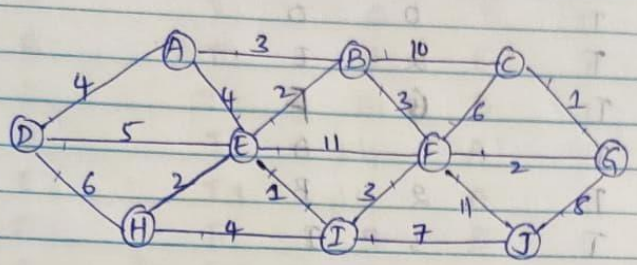
V	Known	dv	Pv
A	T	0	0
B	T	8	E
C	T	6	F
D	T	4	A
E	T	2	B
F	T	3	I
G	F	2	F
H	T	2	E
I	T	1	E
J	T	7	I

Fig:- Table after E, J declared as known.

V	known	dv	Pv
A	T	0	0
B	T	3	A
C	T	1	G
D	T	4	A
E	T	2	B
F	T	3	I
G	T	2	F
H	T	2	E
I	T	1	E
J	T	7	I

Fig:- Table after 'J' declared as known.

Kruskal's Algorithm :-



→ It is a greedy approach, - select the edges in order of smallest weight and add an edge if it doesn't cause a cycle.

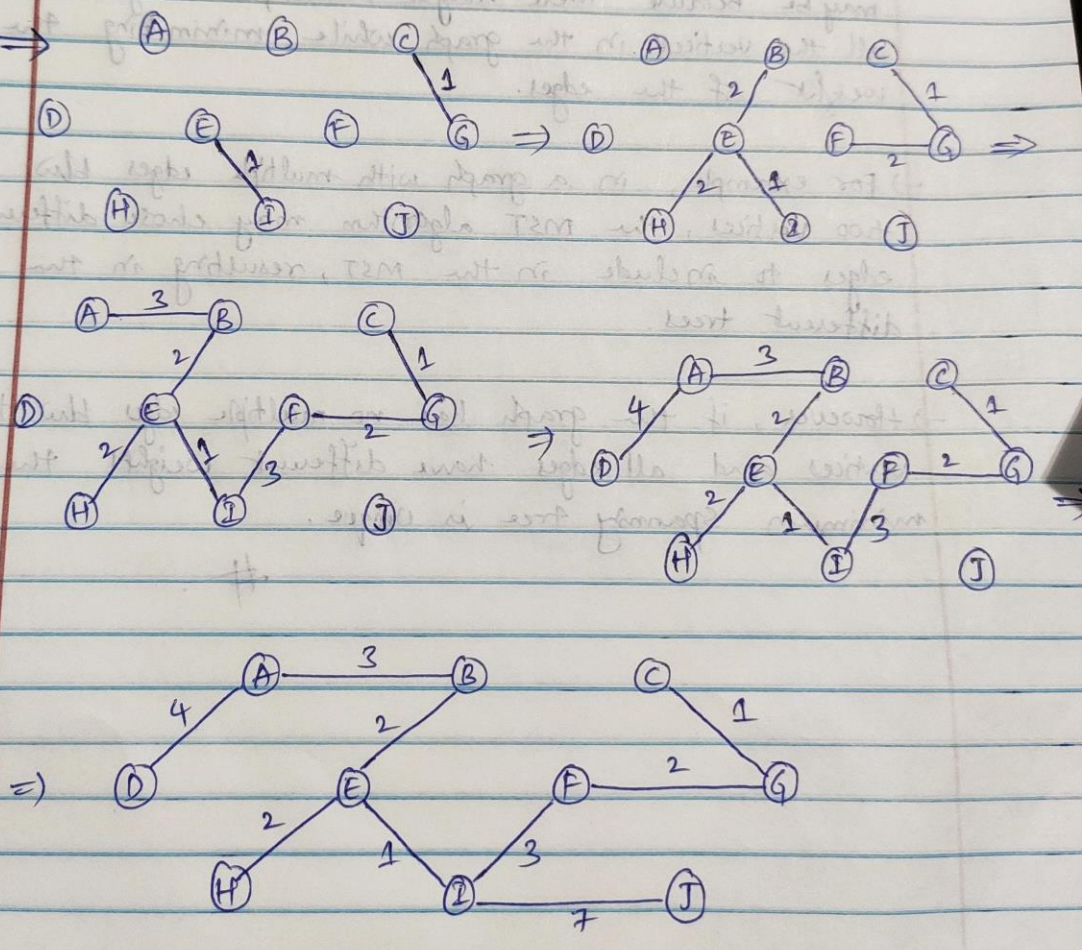
→ The below graphical representation shows Kruskal's Algorithm of each step. Table

Edge	weight	Action
(C,G)	1	Accepted
(E,I)	1	Accepted.
(B,E)	2	Accepted
(E,H)	2	Accepted
(F,G)	2	Accepted.
(A,B)	3	Accepted
(F,I)	3	Accepted.
(B,F)	3	Rejected
(A,D)	4	Accepted
(A,E)	4	Rejected
(H,I)	4	Rejected
(D,E)	5	Rejected
(D,H)	6	Rejected
(F,C)	6	Rejected
(I,J)	7	Accepted
(B,C)	~~10~~	
(G,J)	8	Rejected

Edge	Weight	Action
(B, C)	10	Rejected
(E, F)	11	Rejected
(F, J)	11	Rejected.

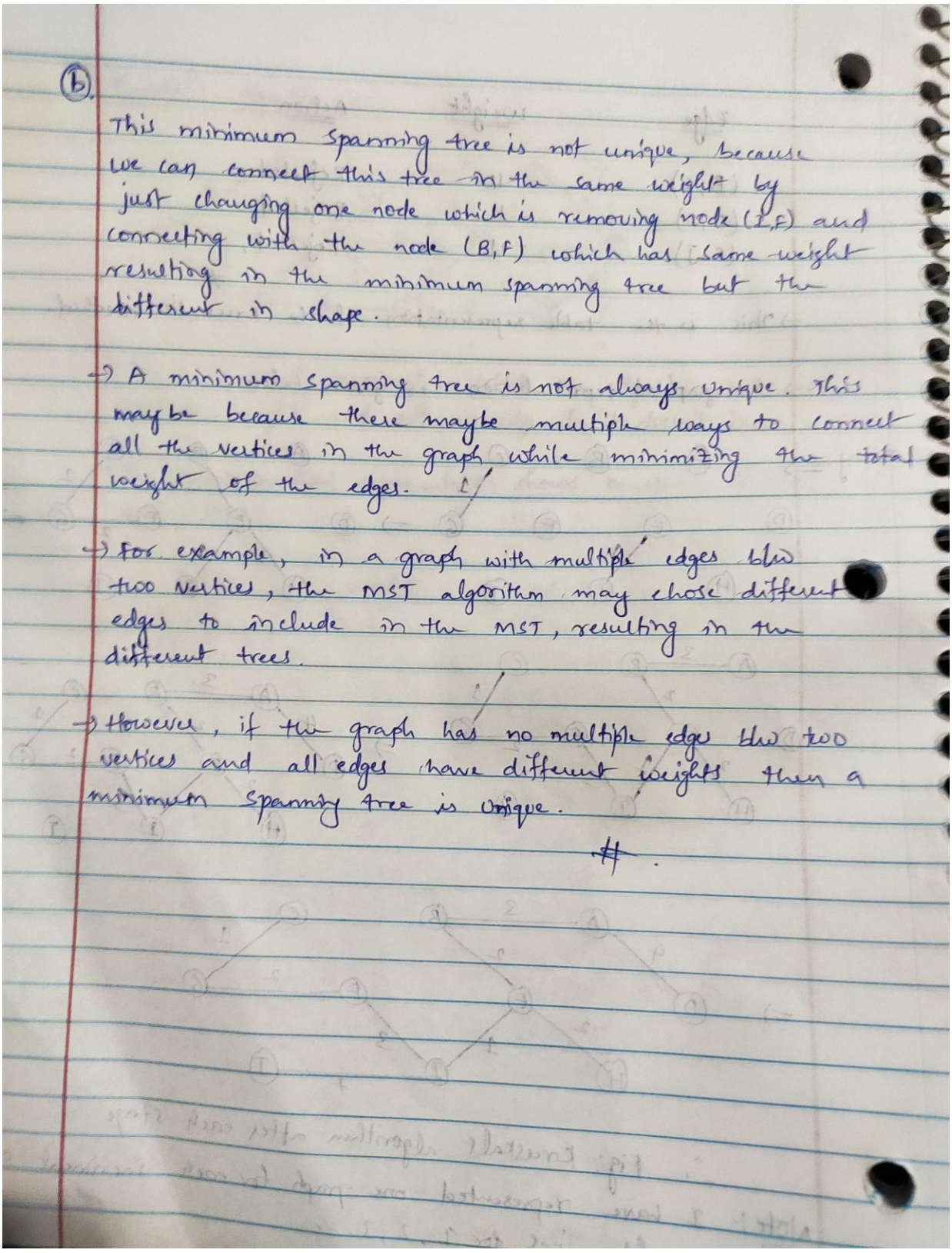
→ This is the table representation of the Kruskal's method.

→ The graphical representation is given below.



∴ Fig: Kruskals algorithm after each stage.

Note:- I have represented one graph for each increment of weight, i.e for 1, 2, 3...



9.39) A graph is *k*-colorable if each vertex can be given one of *k* colors, and no edge connects identically colored vertices. Give a linear-time algorithm to test a graph for two-colorability.

Assume graphs are stored in adjacency list format; you must specify any additional data structures that are needed.

Ans: -

To test a graph for two-colorability (bipartiteness), you can use a linear-time algorithm based on graph coloring. This algorithm involves a depth-first search (DFS) and is known as the "Bipartite Graph Check" algorithm.

1. Pick a starting vertex and assign it a color.
2. For each of its adjacent vertices, assign the opposite color.
3. Repeat steps 1 and 2 until all vertices are colored.
4. If any two adjacent vertices have the same color, then the graph is not two-colorable.
5. Otherwise, the graph is two-colorable.

Here is a Java pseudo code implementation of the algorithm:

// Used the google for some reference in the code.

```
public boolean ColorableGraph(Graph graph) {
  // Initialize a color array to store the colors of the vertices.
  int[] color = new int[graph.getNumVertices()];
  // Initialize all vertices as uncolored.
  for (int i = 0; i < graph.getNumVertices(); i++) {
    color[i] = -1;
  }
  // Start DFS from the first vertex.
  if (!dfs(graph, 0, color)) {
    return false;
  }
  // If all vertices are colored, then the graph is two-colorable.
  return true;
}
```

```
private boolean dfs(Graph graph, int vertex, int[] color) {
  // If the current vertex is already colored, then return true.
  if (color[vertex] != -1) {
    return true;
  }
  // Assign a color to the current vertex.
  color[vertex] = 0;
  // For each of the adjacent vertices, assign the opposite color.
  for (int i = 0; i < graph.getAdjacentVertices(vertex).size(); i++) {
    int adjacentVertex = graph.getAdjacentVertices(vertex).get(i);
    if (!dfs(graph, adjacentVertex, color)) {
      return false;
    }
  }
  // If all adjacent vertices are colored, then return true.
  return true;
}
```

The time complexity of the algorithm is O(N + M), where N is the number of vertices and M is the number of edges in the graph. This is because we iterate over each vertex and each edge once.

Qn) Application of Graph Algorithms: There are N cities and the cost of providing direct flight between any two cities is given. Determine a set of direct flights to offer so that all cities are connected at a minimal cost. You may ignore the direction of flights and assume that the direct flight between any two selected cities is bidirectional. (20 points)

- **Give clear explanations on how this problem can be solved using graph model: What do vertices represent? What do edges represent?**
- **Describe the algorithm used to find the answer.**
- **Analyze the time complexity of your algorithm.**

Ans: -

This problem can be effectively solved using graph theory and graph algorithms.
The vertices in the graph model represent the cities, and the edges represent the direct flights between the cities.
The cost of providing a direct flight between two cities is associated with the weight of the corresponding edge.
To find the set of direct flights to offer so that all cities are connected at a minimal cost, we can use a greedy algorithm known as Kruskal's algorithm. This algorithm starts by sorting the edges in ascending order of their cost.
Then, it iteratively adds the edges to a minimum spanning tree, which is a subset of the edges that connects all the vertices without forming any cycles. The algorithm stops when all the vertices are connected.

Here's a step-by-step explanation of how this problem can be solved using the graph model:

Model the Problem as a Graph: Create a weighted, undirected graph where each city is a vertex, and the cost of providing a direct flight between two cities is the weight of the edge between them.

Minimum Spanning Tree (MST):

Kruskal's Algorithm:
Initialize an empty graph as the MST.
Sort all the edges in non-decreasing order of their weights.
Iterate through the sorted edges and add them to the MST if they don't create a cycle in the MST.
Continue this process until you have (N - 1) edges in the MST, where N is the number of cities.
This forms a spanning tree that connects all cities.

Output: The set of direct flights forming the Minimum Spanning Tree represents the minimal cost set of flights that connect all cities.
The time complexity of Kruskal's algorithm is O (E log E), where E is the number of edges in the graph.
This is because the algorithm needs to sort the edges, which takes O (E log E) time, and then it needs to iterate over the edges, which takes O(E) time.

Vertices: Each vertex in the graph represents a city.
Edges: Each edge in the graph represents a possible direct flight between two cities.

The weight of each edge is the cost of providing a direct flight between the two corresponding cities. **Algorithm:**

_____*_____